

```

# backend/main.py
from fastapi import FastAPI, HTTPException, Depends
from pydantic import BaseModel
from sqlalchemy.orm import Session
import datetime
import uuid

# Imports absolutos para o banco de dados
from database import get_db
from models import TransacaoModel

app = FastAPI(title="API SisCuratela Pro - Backend de Produção")

# =====
# 1. CONTRATOS DE DADOS (PYDANTIC)
# =====
class LoginRequest(BaseModel):
    email: str
    senha: str

class LoginResponse(BaseModel):
    access_token: str
    nome: str
    perfil: str

class LancamentoRequest(BaseModel):
    id_conta: str
    id_usuario: str
    id_micro: str
    data_transacao: str
    valor: float
    descricao: str
    documento_referencia: str | None = None

# =====
# 2. BASE DE USUÁRIOS OFICIAL
# =====
USUARIOS_OFICIAIS = {
    "allandourado@gmail.com": {
        "id": "uuid-allan-001",
        "nome": "Allan Dourado",
        "perfil": "CURADOR_ADMIN",
        "senha": "SenhaForte123!"
    },
    "pereiraitamar2@gmail.com": {
        "id": "uuid-itamar-002",
        "nome": "Itamar Pereira",
        "perfil": "CURADOR_ADMIN",
        "senha": "SenhaForte123!"
    },
    "nelsonf.adv@gmail.com": {
        "id": "uuid-nelson-003",
        "nome": "Dr. Nelson Ferreira",
        "perfil": "ADVOGADO_AUDITOR",
    }
}

```

```

        "senha": "SenhaForte123!"
    }
}
def registrar_audit_log_db(email: str, acao: str, status: str):
    timestamp = datetime.datetime.now()
    print(f"[AUDIT TRAIL POSTGRESQL] {timestamp} | E-MAIL: {email} | AÇÃO: {acao} | STATUS: {status}")

# =====
# 3. ENDPOINTS (ROTAS)
# =====
@app.post("/auth/login", response_model=LoginResponse)
def login_oficial(request: LoginRequest):
    usuario = USUARIOS_OFICIAIS.get(request.email)

    if not usuario or usuario["senha"] != request.senha:
        registrar_audit_log_db(request.email, "LOGIN_FALHA", "Credenciais Inválidas")
        raise HTTPException(status_code=401, detail="E-mail ou senha incorretos.")

    token_seguro = f"jwt_prod_{usuario['nome'].replace(' ', '')}_{datetime.datetime.now().timestamp()}"
    registrar_audit_log_db(request.email, "LOGIN_SUCESSO", f"Acesso autorizado para perfil {usuario['perfil']}")

    return LoginResponse(
        access_token=token_seguro,
        nome=usuario["nome"],
        perfil=usuario["perfil"]
    )

@app.post("/transacoes/novo")
def registrar_lancamento(dados: LancamentoRequest, db: Session = Depends(get_db)):
    if dados.valor <= 0:
        raise HTTPException(status_code=400, detail="O valor da transação deve ser superior a zero.")

    # Trava de Compliance MPDFT
    termos_proibidos = ["diversos", "variados", "gasto geral"]
    if any(termo in dados.descricao.lower() for termo in termos_proibidos):
        raise HTTPException(
            status_code=400,
            detail="Compliance Rejeitado: Descrições genéricas não são permitidas."
        )

    # Prepara a gravação no banco
    nova_transacao = TransacaoModel(
        id_transacao=str(uuid.uuid4()),
        id_conta=dados.id_conta,
        id_usuario=dados.id_usuario,
        id_micro=dados.id_micro,

```

```

        data_transacao=dados.data_transacao,
        valor=dados.valor,
        descricao=dados.descricao,
        documento_referencia=dados.documento_referencia,
        exige_alvara=True if dados.valor > 5000.00 else False
    )

    try:
        db.add(nova_transacao)
        db.commit()
        db.refresh(nova_transacao)
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500, detail=f"Erro ao gravar transação
no banco de dados: {str(e)}")

    return {
        "status": "sucesso",
        "mensagem": "Lançamento financeiro gravado com sucesso no PostgreSQL e
validado pelo compliance.",
        "id_transacao": nova_transacao.id_transacao
    }

```